SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	**Assessment:**	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO5 Statement:	*Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.*		
Student Name:	MANDLA VISWATEJA	**Reg. No.:**	192311399
Section / Batch:	CSE / 2023	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I* (J KARTHIK / 192472136) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J KARTHIK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Constraint-Based Problem Solving

Backtracking, Pruning & Complexity Analysis — Q61 to Q65

Q61. Constraint-Based Circuit Layout Design (NP-Hard Problem)

Arrange n circuit components on a chip area so that total wire length between connected components is minimised, no two components overlap, and every component stays within the chip boundary.

1. Formal Definition of Constraints

1. Boundary constraint: for every component i placed at position (x_i, y_i) with footprint (w_i, h_i), the full footprint must lie inside the chip region [0, W] x [0, H].
2. Non-overlap constraint: for any two components i != j, their bounding rectangles must not intersect, i.e. NOT (x_i < x_j + w_j AND x_j < x_i + w_i AND y_i < y_j + h_j AND y_j < y_i + h_i).
3. Connectivity constraint: a netlist N defines which component pins must be electrically connected; wire length for a net is estimated using Half-Perimeter Wire Length (HPWL) over the components in that net.
4. Objective: minimise total wire length, Sum over all nets of HPWL(net), subject to constraints 1 and 2 being satisfied for every placed component.

2. How the Constraints Interact Spatially / Structurally

- Placing one component shrinks the free placement region for every component placed after it — the non-overlap constraint is therefore history-dependent, not independent per component.
- The boundary constraint interacts with non-overlap: as more components are placed near the edges, the remaining legal area for later components becomes fragmented, which is exactly what backtracking must search over.
- The connectivity (wire-length) constraint couples components that are not necessarily placed consecutively in the search order, since a net can involve components chosen far apart in the recursion — this is why a lower-bound estimate (e.g., minimum spanning tree over net pins) is needed for effective pruning rather than a purely local check.
- Together, the constraints define a spatial CSP: variables = component positions, domains = legal grid sites, and two constraint families (unary: boundary; binary: non-overlap) plus a global optimisation objective (wire length).

3. Backtracking-Based Solution

Components are ordered by decreasing connectivity degree (most-connected first, a classic 'most constrained variable' heuristic) so that costly components are placed early and bad branches are detected sooner. At each recursive call, the next component is tried at every legal grid site; a site is legal only if it satisfies the boundary and non-overlap constraints against all already-placed components.

```
def place_components(components, chip_w, chip_h, grid_sites, netlist):
    order = sorted(components, key=lambda c: -degree(c, netlist))  # most-connected first
    placement = {}
    best = {'layout': None, 'cost': float('inf')}

    def fits(comp, x, y, placement):
        w, h = comp.w, comp.h
        if x < 0 or y < 0 or x + w > chip_w or y + h > chip_h:
            return False                                    # boundary constraint
        for other, (ox, oy) in placement.items():
            if rectangles_overlap(x, y, w, h, ox, oy, other.w, other.h):
                return False                                # non-overlap constraint
        return True

    def backtrack(i, partial_cost):
        if partial_cost >= best['cost']:                    # branch-and-bound pruning
            return
        if i == len(order):
            best['cost'] = partial_cost
            best['layout'] = dict(placement)
```

```
            return
        comp = order[i]
        for (x, y) in grid_sites:
            if fits(comp, x, y, placement):
                placement[comp] = (x, y)
                new_cost = partial_cost + incremental_wirelength(comp, placement, netlist)
                lb = new_cost + lower_bound_remaining(order[i+1:], netlist)
                if lb < best['cost']:                        # prune only if bound is still promising
                    backtrack(i + 1, new_cost)
                del placement[comp]

    backtrack(0, 0)
    return best
```

4. Pruning Strategy

- Overlap pruning: a candidate site is discarded immediately (O(k) rectangle-intersection tests against k already-placed components) before any recursive call is made.
- Branch-and-bound pruning: a lower bound on the remaining wire length (e.g., minimum spanning tree over each unresolved net's pins, or straight-line distance sums) is added to the partial cost; if this optimistic bound already exceeds the best complete layout found so far, the whole subtree is discarded.
- Ordering heuristic (most-connected first): placing highly connected components early produces a strong incumbent solution quickly, which tightens the pruning bound for the rest of the search.
- Symmetry pruning: layouts that are pure translations/rotations/reflections of an already-explored layout are skipped, since they have identical cost.

5. Correctness Justification

The grid of legal sites is finite, so the recursion tree explored by backtrack() is finite and every leaf corresponds to a complete, constraint-satisfying placement of all components.

The overlap and boundary checks in fits() are applied before every recursive call, so no branch below an invalid partial placement is ever explored — every leaf that is reached is guaranteed feasible.

The only pruning applied beyond feasibility is the branch-and-bound cost bound, which discards a subtree only when a provable lower bound on its best possible completion is already worse than a layout already found — so no optimal solution can be lost, only proven-inferior branches are cut.

Consequently, when the search terminates, best['layout'] holds a layout that is feasible (by construction) and optimal (because every unexplored branch was provably no better).

6. Complexity Classification

Aspect	Analysis
Search space size	O(s^n) placements for n components over s legal sites, before pruning
Relation to known NP-hard problems	The problem generalises the Quadratic Assignment Problem (QAP): assign n components to s sites minimising sum of (connection strength x distance). QAP is a classical NP-hard problem.
Reduction sketch	Any QAP instance (facilities, flows, distances) maps directly to a circuit layout instance where facilities are components, flows are net weights, and distances are site-to-site chip distances; solving the layout problem optimally solves the QAP instance.
Conclusion	Since circuit layout contains QAP as a special case, it is at least as hard as QAP, so Circuit Layout Design is NP-Hard.

Because the problem is NP-Hard, no known algorithm solves all instances in polynomial time; the backtracking + branch-and-bound approach is exact but worst-case exponential, which is expected and acceptable for an NP-Hard problem — pruning reduces the practical runtime without changing the worst-case class.

Q62. Minimum Dominating Edge Set Problem (NP-Complete Problem)

Given a graph G = (V, E), find a minimum-size subset D ⊆ E such that every edge in E is either in D or shares an endpoint (is adjacent) with some edge in D.

1. Formal Definition of Constraints

1. Domination constraint: for every edge e = (u, v) in E, either e is in D, or there exists an edge e' in D such that e' shares vertex u or vertex v with e (e and e' are adjacent).
2. Subset constraint: D is a subset of E — no fractional or repeated selection of edges.
3. Minimality objective: among all edge sets D satisfying the domination constraint, |D| must be as small as possible.
4. (Decision form used for NP-completeness) Given integer k, does there exist a dominating edge set D with |D| <= k?

2. How the Constraints Interact Spatially / Structurally

- Selecting one edge dominates every edge incident to either of its two endpoints, so high-degree edges dominate large 'neighbourhoods' of E — the constraint is inherently non-local: one choice affects many other edges at once.
- Overlap between the domination footprints of two chosen edges is wasted coverage, so the constraints push toward choosing edges whose neighbourhoods overlap the undominated region as little as possible (a set-cover-like interaction).
- It is a known structural fact that every maximal matching is a dominating edge set, and the smallest maximal matching equals the minimum edge dominating set in size — so the domination constraint is tightly linked to the independence (non-adjacency) constraint used in matchings.
- Pendant edges (incident to a degree-1 vertex) can only be dominated by themselves or their single neighbour edge, which forces certain inclusions and sharply restricts the feasible search space around low-degree vertices.

3. Backtracking-Based Solution

Edges are processed in an order that favours high total endpoint-degree first (so early inclusion decisions dominate more of the graph, producing a strong incumbent bound quickly). At each edge, the algorithm branches into 'include in D' and 'exclude from D', maintaining a bitset of which edges are already dominated.

```
def min_edge_dominating_set(edges, adjacency):
    order = sorted(edges, key=lambda e: -(degree(e[0]) + degree(e[1])))
    best = {'set': None, 'size': float('inf')}

    def dominates(e1, e2):
        return e1 == e2 or set(e1) & set(e2)          # share a vertex or identical edge

    def backtrack(i, chosen, dominated):
        if len(chosen) >= best['size']:                # bound pruning
            return
        if len(dominated) == len(edges):
            best['set'], best['size'] = list(chosen), len(chosen)
            return
        if i == len(order):
            return                                     # ran out of edges, infeasible branch

        e = order[i]
        newly_dominated = {f for f in edges if f not in dominated and dominates(e, f)}

        if newly_dominated:                            # skip edges that add no new domination
            chosen.append(e)
            backtrack(i + 1, chosen, dominated | newly_dominated)
            chosen.pop()

        # exclude e, but only if remaining edges can still dominate what e would have covered
        if can_still_dominate(order[i+1:], dominated, edges):
            backtrack(i + 1, chosen, dominated)

    backtrack(0, [], set())
    return best
```

4. Pruning Strategy

- Redundancy pruning: an edge that dominates no currently-undominated edge is never added to D (it cannot improve feasibility, only waste budget).
- Bound pruning: if |chosen| already equals or exceeds the best solution size found so far, the branch is abandoned, since it cannot improve on the incumbent.
- Forced-inclusion pruning: pendant / degree-1-incident edges that are the only possible dominator of themselves are included immediately rather than branched on, collapsing two branches into one.
- Feasibility pruning on exclusion: an edge is only excluded if some other still-available edge can dominate everything it would have dominated; otherwise that branch is infeasible and cut immediately.

5. Correctness Justification

Every edge is explicitly given an include/exclude branch, so the recursion tree implicitly enumerates every subset of E — nothing is missed except subsets pruned for a proven reason.

The base case only accepts a solution when the dominated set equals the full edge set E, so every accepted D satisfies the domination constraint exactly as defined.

Redundancy and feasibility pruning never remove a branch that could lead to a smaller valid D than the current best — they only remove edges that add no coverage, or exclude edges only when coverage is still achievable by remaining edges — so completeness with respect to optimal solutions is preserved.

Bound pruning removes a branch only once its edge count already matches or exceeds a verified feasible solution, so it can only ever discard provably non-improving branches.

6. Complexity Classification

Aspect	Analysis
Membership in NP	A candidate D can be verified in O(|E|^2) time by checking, for every edge, whether it is dominated — so the decision problem is in NP.
NP-hardness	Minimum Edge Dominating Set is equivalent in size to Minimum Maximal Matching, which was proven NP-complete by Yannakakis and Gavril (1980) via reduction from Vertex Cover on graphs of maximum degree 3.
Reduction sketch	A minimum vertex cover instance on a bounded-degree graph can be transformed (by edge subdivision / gadget construction) into a graph where a minimum maximal matching / edge dominating set of size k exists exactly when the original graph has a vertex cover of a corresponding size.
Conclusion	Because the problem is in NP and is at least as hard as a known NP-complete problem, the Minimum Dominating Edge Set decision problem is NP-Complete.

Being NP-Complete means both a valid solution is quick to check and the search for an optimal one is (believed to be) exponential in the worst case — the backtracking algorithm above is exact and complete but not polynomial-time, which matches the expected complexity class.

Q63. Constraint-Based DNA Sequence Assembly (NP-Hard Problem)

Given a set of DNA fragments (reads), reconstruct the shortest possible complete sequence by ordering and overlapping fragments consistently, so the assembled sequence explains every fragment.

1. Formal Definition of Constraints

1. Overlap constraint: fragment b may be appended after fragment a only if a suffix of a matches a prefix of b with overlap length >= a minimum threshold t (possibly allowing up to e mismatches).
2. Usage constraint: each fragment is used exactly once in the assembly (a Hamiltonian-path style

ordering over the fragment set).

3. Consistency constraint: wherever two or more fragments overlap in the final assembly, the overlapping region must contain identical base-pair content across all of them — no contradictory bases in a shared region.
4. Minimisation objective: choose the ordering and overlaps that minimise the length of the final assembled superstring (equivalently, maximise total overlap used).

2. How the Constraints Interact Spatially / Structurally

- The overlap constraint defines a directed weighted graph over fragments (an edge a -> b exists with weight equal to maximum valid overlap length), so the assembly problem becomes a search for a maximum-weight Hamiltonian path in that graph.
- Fixing an early ordering decision removes options later: once fragment a is placed at position i, only fragments with valid overlap against a's suffix are candidates for position i+1, so partial assemblies prune the remaining search space themselves.
- The consistency constraint interacts with the overlap constraint transitively: a chain of pairwise-valid overlaps must not still produce mismatched bases when three or more fragments overlap in the same region, so local pairwise checks are necessary but not always sufficient — a running consensus string must be checked as fragments are added.
- The usage constraint (each fragment exactly once) means the problem is combinatorial over permutations, not subsets, which is what makes it structurally identical to the Hamiltonian Path / Travelling Salesman family rather than a simpler covering problem.

3. Backtracking-Based Solution

An overlap graph is precomputed: an edge from fragment a to fragment b is added only if the overlap constraint is satisfiable, weighted by the maximum overlap length. Backtracking then searches for the fragment ordering (Hamiltonian path) that maximises total overlap, which minimises the final assembled length.

```
def assemble(fragments, min_overlap):
    overlap_graph = build_overlap_graph(fragments, min_overlap)  # edge weight = max valid overlap
    best = {'order': None, 'total_overlap': -1}

    def backtrack(path, used, total_overlap):
        bound = total_overlap + optimistic_remaining_overlap(path[-1], used, overlap_graph)
        if bound <= best['total_overlap']:              # branch-and-bound pruning
            return
        if len(path) == len(fragments):
            if total_overlap > best['total_overlap']:
                best['order'], best['total_overlap'] = list(path), total_overlap
            return

        last = path[-1]
        for nxt, ov in overlap_graph[last]:              # only fragments with valid overlap
            if nxt not in used and consistent(path, nxt, ov):
                path.append(nxt); used.add(nxt)
                backtrack(path, used, total_overlap + ov)
                path.pop(); used.discard(nxt)

    for start in fragments:                              # try every possible starting fragment
        backtrack([start], {start}, 0)

    return build_sequence(best['order'], overlap_graph)
```

4. Pruning Strategy

- Feasibility pruning: only fragments with a valid, threshold-satisfying overlap against the current path's last fragment are tried as the next step — infeasible extensions are never generated.
- Consistency pruning: an extension is rejected immediately if the overlapping region would conflict with bases already fixed by earlier fragments, avoiding wasted exploration of contradictory assemblies.
- Branch-and-bound pruning: an optimistic upper bound on the remaining achievable overlap (e.g., the best single overlap available from the current fragment to any unused fragment, summed heuristically) is added to the running total; if this bound cannot beat the best complete assembly found so far, the branch is cut.

- Visited-set pruning: the 'used' set prevents any fragment from being placed twice, collapsing the search from all sequences to only valid permutations.

5. Correctness Justification

Every backtracking call extends a valid partial ordering of distinct, unused fragments, so any path that reaches full length (len(path) == len(fragments)) is a complete, non-repeating fragment ordering — respecting the usage constraint by construction.

The overlap and consistency checks are enforced before a fragment is appended, so every completed path also respects the overlap and consistency constraints, guaranteeing feasibility of every accepted assembly.

The bound pruning only discards a branch when its best possible future total overlap still cannot exceed a total overlap already achieved by a complete, valid assembly — so no branch capable of producing a better assembly is ever removed.

Trying every fragment as a starting point ensures the search is not biased toward a single (possibly suboptimal) origin, preserving global optimality of the returned assembly.

6. Complexity Classification

Aspect	Analysis
Search space size	O(n!) possible fragment orderings for n fragments, before pruning
Relation to known NP-hard problems	Maximising total overlap over an ordering of fragments is exactly the Shortest Common Superstring (SCS) problem, a well-studied NP-hard problem.
Reduction sketch	Any Hamiltonian Path instance on a graph can be encoded as a fragment-overlap instance (fragments as gadget strings, overlap only exists along the original graph's edges); a Hamiltonian path exists in the graph exactly when an assembly using all fragments with full overlap exists, showing the assembly problem is at least as hard as Hamiltonian Path.
Conclusion	Since DNA assembly generalises the SCS / Hamiltonian Path family of problems, Constraint-Based DNA Sequence Assembly is NP-Hard.

Real assemblers (e.g., de Bruijn graph based tools) avoid the full NP-hard formulation by working with k-mers instead of whole-fragment permutations, trading exactness for polynomial-time heuristics — the backtracking solution here is the exact but exponential formulation appropriate for small instances or as a correctness baseline.

Q64. Multi-Dimensional Knapsack Problem (NP-Hard Problem)

Given n items, each with a profit and resource consumption across m independent dimensions (e.g., weight, volume, budget), select a subset of items maximising total profit without exceeding any dimension's capacity.

1. Formal Definition of Constraints

1. Per-dimension capacity constraint: for every dimension d in {1, ..., m}, the sum of w[i][d] over all selected items i must not exceed capacity C_d.
2. Binary selection constraint: each item's selection variable x_i is in {0, 1} — an item is either fully taken or not taken (0/1 knapsack, no fractional selection).
3. Objective: maximise Sum of p_i * x_i over all i, subject to all m capacity constraints holding simultaneously.

2. How the Constraints Interact Spatially / Structurally

- Because every selected item consumes resources in all m dimensions at once, a selection that is feasible in dimension 1 can still be infeasible in dimension 2 — feasibility is the intersection of m separate knapsack constraints, not their union.
- This intersection shrinks the feasible region much faster than any single dimension would suggest: an item combination that looks 'cheap' overall may still be blocked by just one tight dimension.
- The interaction destroys the pseudo-polynomial dynamic programming approach that works for the single-dimension 0/1 knapsack: the DP state would need to track remaining capacity in all m dimensions simultaneously, giving a state space of O(n * C_1 * C_2 * ... * C_m), which grows exponentially in m.
- Item dominance becomes multi-dimensional too: an item is only 'dominated' (safely removable) if another item has both a higher or equal profit AND lower or equal consumption in every single dimension — a much stricter condition than in the 1-D case, so fewer items can be pruned by dominance alone.

3. Backtracking-Based Solution

Items are sorted by a profit-to-aggregate-resource ratio to try promising items first and obtain a strong incumbent quickly. At each item, the algorithm branches into 'take' and 'skip'; the 'take' branch is only explored if it does not violate any of the m capacity constraints.

```
def multi_dim_knapsack(items, capacities):                # capacities = [C_1, ..., C_m]
    order = sorted(items, key=lambda it: -it.profit / sum(it.weights))
    best = {'selection': None, 'profit': 0}

    def fits(usage, item, capacities):
        return all(usage[d] + item.weights[d] <= capacities[d] for d in range(len(capacities)))

    def bound(i, cur_profit, usage):                      # optimistic fractional-relaxation bound
        return cur_profit + fractional_upper_bound(order[i:], usage, capacities)

    def backtrack(i, cur_profit, usage, chosen):
        if i == len(order):
            if cur_profit > best['profit']:
                best['profit'], best['selection'] = cur_profit, list(chosen)
            return
        if bound(i, cur_profit, usage) <= best['profit']:  # branch-and-bound pruning
            return

        item = order[i]
        if fits(usage, item, capacities):                 # feasibility pruning
            chosen.append(item)
            new_usage = [usage[d] + item.weights[d] for d in range(len(capacities))]
            backtrack(i + 1, cur_profit + item.profit, new_usage, chosen)
            chosen.pop()

        backtrack(i + 1, cur_profit, usage, chosen)       # skip branch, always valid

    backtrack(0, 0, [0] * len(capacities), [])
    return best
```

4. Pruning Strategy

- Feasibility pruning: the 'take' branch is generated only when every one of the m capacity constraints would still hold, so infeasible partial selections are never explored.
- Branch-and-bound pruning: an optimistic bound from the fractional (LP-relaxed) version of the remaining sub-problem is computed at every node; if even this best-case bound cannot beat the current best complete profit, the whole subtree is discarded.
- Dominance pruning: an item that is profit-dominated in every dimension by another item is removed from consideration before search begins, shrinking n.
- Ratio-based ordering: sorting by profit-to-aggregate-resource ratio produces good feasible solutions early, tightening the incumbent bound and making branch-and-bound pruning effective sooner in the search.

5. Correctness Justification

Every item is given exactly one 'take' branch and one 'skip' branch at each recursive level, so the recursion tree implicitly enumerates all 2^n subsets of items.

The fits() check is applied before entering any 'take' branch, so every branch that is actually explored maintains a resource usage vector that never exceeds any capacity — every leaf reached therefore represents a fully feasible selection.

Branch-and-bound pruning uses a provably optimistic bound (the fractional relaxation always over-estimates the true achievable profit of a 0/1 selection), so a branch is discarded only when it is mathematically impossible for it to beat the current best — no optimal solution can be lost this way.

Because both branches (take/skip) are always considered whenever the bound is still promising, and the base case captures every complete, feasible selection, the algorithm returns the true maximum-profit feasible selection.

6. Complexity Classification

Aspect	Analysis
Search space size	O(2^n) subsets before pruning, independent of m
Relation to known NP-hard problems	The single-dimension (m = 1) 0/1 Knapsack Problem is already (weakly) NP-hard via reduction from Subset-Sum; the multi-dimensional version contains the 1-D problem as the special case m = 1.
Strong NP-hardness for m >= 2	Unlike the 1-D case (which admits a pseudo-polynomial DP in O(n * C)), no pseudo-polynomial algorithm is known for m >= 2; the multi-dimensional case can encode problems such as 3-Dimensional Matching, making it strongly NP-hard.
Conclusion	Multi-Dimensional Knapsack is NP-Hard, and for m >= 2 it is strongly NP-hard (no pseudo-polynomial-time algorithm exists unless P = NP).

The distinction between 'NP-hard' and 'strongly NP-hard' matters here: the 1-D knapsack can still be solved efficiently for moderate capacities via DP, but the multi-dimensional version cannot — this is exactly why an exact solution needs a combinatorial backtracking/branch-and-bound search rather than a DP table.

Q65. Constraint-Based Password Cracking (Backtracking Problem)

Determine a password of fixed length L over an alphabet, given constraints on which characters are allowed at which positions, required character-class composition, and pattern rules, verified against a target check (e.g., a hash match).

1. Formal Definition of Constraints

1. Position-domain constraint: the character at position i must belong to an allowed set Sigma_i subset of Sigma (e.g., position 0 must be uppercase, positions 1-6 may be alphanumeric).
2. Composition constraint: the completed password must contain at least k1 digits, k2 uppercase letters, k3 special symbols, etc., counted over the whole string.
3. Pattern constraint: local structural rules such as 'no two adjacent characters are identical' or 'no forbidden substring appears'.
4. Verification constraint: a completed candidate string s is only accepted if it passes an external check, verify(s) (e.g., hash(s) == target_hash, or all rules re-confirmed).

2. How the Constraints Interact Spatially / Structurally

- Position-domain constraints act locally and immediately shrink the branching factor at each recursion level to |Sigma_i| instead of the full alphabet size, so they compound multiplicatively across positions.
- Pattern constraints (e.g., no repeated adjacent character) only depend on the immediately preceding character, so they can be checked with O(1) work at each step, pruning invalid branches the instant they are formed rather than only at the end.

- Composition constraints are global and cumulative: they must be tracked as running counters (digits seen so far, uppercase seen so far, etc.) throughout the recursion, and they interact with position constraints because remaining positions limit how much of a deficit can still be repaired — this is what enables the strongest pruning in this problem.
- The verification constraint is only checked once a full-length candidate is built, but partial verification (e.g., a known prefix hash, if available) can be folded in earlier to prune whole subtrees before reaching full length.

3. Backtracking-Based Solution

The password is built one position at a time via depth-first backtracking. At each position, only characters from that position's allowed domain are tried; running counters for each required character class are updated and checked for feasibility before recursing further.

```
def crack_password(L, domains, required_counts, forbid_repeat, verify):
    # domains[i]        -> allowed character set for position i
    # required_counts   -> dict like {'digit': 2, 'upper': 1, 'symbol': 1}
    password = [None] * L
    counts = {k: 0 for k in required_counts}

    def remaining_feasible(pos, counts):
        remaining_positions = L - pos
        deficit = sum(max(0, required_counts[k] - counts[k]) for k in required_counts)
        return deficit <= remaining_positions              # composition feasibility pruning

    def backtrack(pos, counts):
        if not remaining_feasible(pos, counts):
            return None
        if pos == L:
            candidate = ''.join(password)
            if all(counts[k] >= required_counts[k] for k in required_counts) and verify(candidate):
                return candidate
            return None

        for ch in domains[pos]:                            # only this position's legal domain
            if forbid_repeat and pos > 0 and ch == password[pos - 1]:
                continue                                   # pattern constraint pruning
            password[pos] = ch
            new_counts = update_counts(counts, ch)
            result = backtrack(pos + 1, new_counts)
            if result:
                return result
            password[pos] = None
        return None

    return backtrack(0, counts)
```

4. Pruning Strategy

- Domain pruning: only characters in Sigma_i are ever tried at position i, instead of the full alphabet — this alone can reduce the branching factor by an order of magnitude per position.
- Pattern pruning: adjacency rules (e.g., no repeated character) are checked in O(1) before recursing, eliminating invalid branches at the moment they would be created.
- Composition feasibility pruning: at every position, if the remaining required character-class deficit exceeds the number of positions left to fill, the branch is abandoned immediately — this is the strongest cut, since it can eliminate huge suffix subtrees in one check.
- Early-exit on first success: the recursion returns as soon as one valid, verified candidate is found, avoiding exploration of the rest of the search tree once the goal is met.

5. Correctness Justification

Every recursive call only ever places a character that belongs to the correct position's domain and does not violate the local pattern rule, so every fully built candidate automatically satisfies constraints 1 and 3.

The composition requirement is explicitly re-checked at the base case (pos == L) in addition to the incremental feasibility pruning, so a candidate is only returned if it truly satisfies constraint 2 in full, not just 'still possibly satisfiable'.

The composition feasibility pruning removes a branch only when it is mathematically impossible to reach the required counts with the remaining positions — a proven infeasibility, not a heuristic guess — so no

valid password is ever skipped.

Because verify() is checked before any candidate is accepted, the algorithm never returns a false positive, and because the search is a complete depth-first traversal of the (pruned) domain tree, it will find a satisfying password whenever one exists within the given constraints.

6. Complexity Classification

Aspect	Analysis
Search space size (unconstrained)	O(|Sigma|^L) without any constraints
Search space size (constrained)	O(product of |Sigma_i| for i = 1..L), typically far smaller once position, pattern, and composition pruning are applied
Relation to NP-hardness	Deciding satisfiability of an arbitrary set of positional/compositional constraints is a special case of Constraint Satisfaction (CSP), and general CSP satisfiability is NP-complete (it contains Boolean SAT as a special case, via constraints framed as clauses over character choices).
Practical note	For the fixed, simple rule types described here (per-position domains, counts, local patterns), the composition-feasibility check keeps the search efficient in practice even though the general CSP formulation is NP-complete in the worst case.

Password cracking under explicit rules is best framed as a constraint-satisfaction search rather than an optimisation problem: verifying a candidate is cheap (polynomial), but finding one is, in the worst case (arbitrary constraint sets), as hard as general CSP/SAT — the backtracking search above is exact and will terminate, but its worst-case running time remains exponential in L when the constraints are weak or absent.